

SQL Mastery Cheat Sheet

Beginner → Intermediate → Advanced → Expert

A single reference covering everything from your first SELECT to window functions, query optimization, and interview patterns — built to be useful on day one of any data role, at any company.

BEGINNER

INTERMEDIATE

ADVANCED

EXPERT

REFERENCE

Works across MySQL · PostgreSQL · SQL Server · Snowflake · BigQuery

(syntax differences are called out inline where they matter)

Contents

- 01 Beginner — Core Query Anatomy
- 02 Beginner — Filtering, Sorting & NULLs
- 03 Beginner — Joins
- 04 Beginner — Aggregation & Grouping
- 05 Intermediate — Subqueries & CTEs
- 06 Intermediate — Set Operations & CASE Logic
- 07 Intermediate — Data Modification (DML) & Data Types
- 08 Advanced — Window Functions
- 09 Advanced — Indexes, EXPLAIN & Query Optimization
- 10 Advanced — Transactions, Views & Stored Procedures
- 11 Expert — Common Interview Patterns
- 12 Reference — Functions, Operators & Quick Lookup

TIP: This sheet is dense by design. Skim the colored level tags (Beginner / Intermediate / Advanced / Expert / Reference) to jump to your level, and keep the Reference section (12) open while you work — it's the fastest lookup for functions and syntax.

Every SQL query is read by the engine in a different order than you write it. Understanding this **logical execution order** is the single most important beginner concept — it explains why you can't use a column alias in WHERE, but you can in ORDER BY.

```
SELECT column_list -- 5. choose & shape output columns
FROM table_name -- 1. identify source table(s)
JOIN other_table -- 2. combine rows from other tables
ON join_condition
WHERE row_filter -- 3. filter individual rows
GROUP BY grouping_columns -- 4. collapse rows into groups
HAVING group_filter -- 5b. filter groups (after GROUP BY)
ORDER BY sort_columns -- 6. sort the final result
LIMIT row_count; -- 7. cap the number of rows returned
```

Logical order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Written order is different — this mismatch trips up almost every beginner at least once.

Minimal working query

```
SELECT first_name, last_name, salary
FROM employees
WHERE department = 'Engineering'
ORDER BY salary DESC
LIMIT 10;
```

Aliasing (make output & queries readable)

```
SELECT e.first_name AS fname,
e.salary * 12 AS annual_salary
FROM employees AS e;
```

AS is optional for column aliases in most engines (SELECT e.salary total) but always required for table aliases in some (use it everywhere for clarity).

Comparison & logical operators

Operator	Meaning	Example
=, !=, <>	equals / not equals	status = 'active'
<, >, <=, >=	numeric / date comparison	hire_date >= '2024-01-01'
AND / OR / NOT	combine conditions	age > 18 AND country = 'IN'
BETWEEN	inclusive range	salary BETWEEN 50000 AND 90000
IN (...)	match any in a list	city IN ('Chennai', 'Pune')
LIKE / ILIKE	pattern match (ILIKE = case-insensitive, PG)	name LIKE 'A%'
IS NULL / IS NOT NULL	null checks (never = NULL)	phone IS NULL

Wildcard patterns for LIKE

Pattern	Matches
'A%'	Starts with A
'%son'	Ends with son
'%an%'	Contains 'an' anywhere
'_at'	Exactly 3 chars, ending in 'at' (_ = any single char)

NULL handling — the #1 beginner trap

NULL means “unknown,” not zero or empty. Any arithmetic or comparison with NULL returns NULL, not true/false — so **WHERE column = NULL** never matches anything, even NULL rows.

```
-- Wrong: this returns zero rows, always
SELECT * FROM orders WHERE shipped_date = NULL;

-- Correct
SELECT * FROM orders WHERE shipped_date IS NULL;

-- Coalesce to a default value
SELECT COALESCE(shipped_date, 'Not Shipped') FROM orders;

-- NULLIF: return NULL if two values match (avoid div-by-zero)
SELECT revenue / NULLIF(units_sold, 0) FROM sales;
```

Sorting & de-duplicating

```
SELECT DISTINCT department FROM employees;

SELECT name, salary
FROM employees
ORDER BY department ASC, salary DESC; -- multi-column sort

SELECT * FROM employees
ORDER BY salary DESC
LIMIT 10 OFFSET 20; -- pagination (skip 20, take 10)
```

Joins combine rows from two or more tables based on a related column. Picking the wrong join type is the most common source of silently wrong row counts in real reports.

Join Type	Returns
INNER JOIN	Only rows with a match in BOTH tables
LEFT JOIN	All rows from left table + matches from right (NULL if none)
RIGHT JOIN	All rows from right table + matches from left (NULL if none)
FULL OUTER JOIN	All rows from both sides, matched where possible
CROSS JOIN	Cartesian product — every row × every row
SELF JOIN	A table joined to itself (e.g., employee → manager)

Syntax

```
SELECT o.order_id, c.customer_name
FROM orders AS o
INNER JOIN customers AS c
ON o.customer_id = c.customer_id;

-- Customers with NO orders (anti-join pattern)
SELECT c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Self join: employee -> their manager's name
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

TIP: A LEFT JOIN followed by WHERE right_table.col IS NULL is the standard “anti-join” pattern for finding rows that DON'T have a match — memorize it, it comes up constantly in real work and interviews.

Multiple joins & join order

```
SELECT o.order_id, c.customer_name, p.product_name, oi.quantity
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id;
```

Filter as early as possible and join on indexed columns — join order rarely matters for correctness, but always check EXPLAIN if a multi-join query feels slow (see Section 09).

Aggregate functions

Function	Purpose
COUNT(*) / COUNT(col)	Row count / non-null value count
SUM(col)	Total of numeric column
AVG(col)	Average value
MIN(col) / MAX(col)	Smallest / largest value
COUNT(DISTINCT col)	Count of unique values
STRING_AGG / GROUP_CONCAT	Concatenate group values into one string

GROUP BY and HAVING

GROUP BY collapses rows sharing the same value into a single group so aggregate functions can run per group. **HAVING filters groups after aggregation** — WHERE cannot reference aggregate results, this is why HAVING exists.

```
SELECT department, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM employees
WHERE status = 'active' -- filters rows BEFORE grouping
GROUP BY department
HAVING AVG(salary) > 60000 -- filters groups AFTER aggregation
ORDER BY avg_salary DESC;
```

RULE: Every non-aggregated column in SELECT must appear in GROUP BY (in most engines this is enforced; MySQL is looser but it's still bad practice to rely on that).

ROLLUP / CUBE (subtotal rows)

```
SELECT department, job_title, SUM(salary)
FROM employees
GROUP BY ROLLUP (department, job_title);
-- Produces subtotal rows per department, plus a grand total row
```

Subquery types

Type	Where used	Notes
Scalar	Returns one value	Usable anywhere a single value fits
Row/Column	Returns one row or column	Used with IN, ANY, ALL
Table	Returns a full result set	Used in FROM as a derived table
Correlated	References the outer query	Re-evaluated per outer row — can be slow

```
-- Scalar subquery
SELECT name, salary,
(SELECT AVG(salary) FROM employees) AS company_avg
FROM employees;

-- Subquery with IN
SELECT name FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE region = 'APAC');

-- Correlated subquery: employees earning above THEIR dept average
SELECT e.name, e.salary, e.department
FROM employees e
WHERE e.salary > (
SELECT AVG(e2.salary) FROM employees e2
WHERE e2.department = e.department
);
```

Common Table Expressions (CTEs) — readable, named subqueries

```
WITH dept_avg AS (
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
)
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_salary;
```

Recursive CTE (hierarchies, org charts, graphs)

```
WITH RECURSIVE org_chart AS (
SELECT employee_id, name, manager_id, 1 AS level
FROM employees WHERE manager_id IS NULL -- anchor: the CEO
UNION ALL
SELECT e.employee_id, e.name, e.manager_id, oc.level + 1
FROM employees e
JOIN org_chart oc ON e.manager_id = oc.employee_id -- recursive step
)
SELECT * FROM org_chart ORDER BY level;
```

CTEs vs subqueries: functionally similar in most modern engines, but CTEs are named and reusable within the query, which makes complex logic far easier to read and debug.

Set operators

Operator	Behavior
UNION	Combines results, removes duplicate rows
UNION ALL	Combines results, keeps duplicates (faster — no dedup step)
INTERSECT	Rows present in BOTH result sets
EXCEPT / MINUS	Rows in first set but NOT in second

All set operators require the same number of columns with compatible types in each SELECT, in the same order.

```
SELECT customer_id FROM online_orders
UNION
SELECT customer_id FROM store_orders; -- unique customers, either channel

SELECT customer_id FROM online_orders
INTERSECT
SELECT customer_id FROM store_orders; -- customers who used BOTH channels
```

CASE — conditional logic inside SQL

```
SELECT
order_id,
amount,
CASE
WHEN amount >= 1000 THEN 'High'
WHEN amount >= 100 THEN 'Medium'
ELSE 'Low'
END AS order_tier
FROM orders;

-- CASE inside an aggregate = conditional counting (very common pattern)
SELECT
COUNT(CASE WHEN status = 'delivered' THEN 1 END) AS delivered_count,
COUNT(CASE WHEN status = 'cancelled' THEN 1 END) AS cancelled_count
FROM orders;
```

TIP: COUNT(CASE WHEN ... THEN 1 END) is the standard way to pivot categories into columns without a dedicated PIVOT clause — works in every SQL dialect.

INSERT, UPDATE, DELETE, UPSERT

```

INSERT INTO employees (name, department, salary)
VALUES ('Asha Rao', 'Engineering', 85000);

UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Engineering';

DELETE FROM employees
WHERE status = 'terminated' AND last_active < '2023-01-01';

-- Upsert (PostgreSQL syntax) – insert, or update if it already exists
INSERT INTO inventory (sku, quantity)
VALUES ('SKU123', 50)
ON CONFLICT (sku) DO UPDATE SET quantity = inventory.quantity + 50;

```

SAFETY: Always run UPDATE/DELETE logic as a SELECT with the same WHERE clause first, to see exactly which rows will be affected before you commit.

Common data types

Category	Types
Integers	INT, SMALLINT, BIGINT
Decimals	DECIMAL(p,s) / NUMERIC — exact; FLOAT/DOUBLE — approximate
Text	VARCHAR(n), TEXT, CHAR(n)
Date/Time	DATE, TIME, TIMESTAMP, TIMESTAMPTZ
Boolean	BOOLEAN (or TINYINT(1) in MySQL)
Semi-structured	JSON / JSONB (Postgres), STRUCT/ARRAY (BigQuery)

Use DECIMAL/NUMERIC for money — never FLOAT — to avoid rounding errors in financial calculations.

Table creation & constraints

```

CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  department_id INT REFERENCES departments(department_id),
  salary DECIMAL(10,2) CHECK (salary >= 0),
  email VARCHAR(255) UNIQUE,
  hired_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Window functions calculate a value across a set of rows related to the current row — **without collapsing rows like GROUP BY does**. This is the single highest-leverage advanced topic for analytics/data roles.

```
<function>(...) OVER (
PARTITION BY column -- optional: reset calculation per group
ORDER BY column -- optional: defines row order within partition
[frame clause] -- optional: ROWS/RANGE BETWEEN ...
)
```

Ranking functions

Function	Behavior on ties
ROW_NUMBER()	Unique sequential number, no ties (arbitrary tiebreak)
RANK()	Same rank for ties, gaps after (1,2,2,4)
DENSE_RANK()	Same rank for ties, no gaps (1,2,2,3)
NTILE(n)	Splits rows into n roughly equal buckets

```
-- Top 3 earners PER department
SELECT name, department, salary,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
FROM employees
QUALIFY rnk <= 3; -- Snowflake/BigQuery shortcut
-- Standard SQL: wrap the whole thing in a subquery/CTE and WHERE rnk <= 3
```

Offset functions — LAG / LEAD

```
SELECT order_date, revenue,
LAG(revenue) OVER (ORDER BY order_date) AS prev_day_revenue,
LEAD(revenue) OVER (ORDER BY order_date) AS next_day_revenue,
revenue - LAG(revenue) OVER (ORDER BY order_date) AS day_over_day
FROM daily_sales;
```

Running totals & moving averages

```
SELECT order_date, revenue,
SUM(revenue) OVER (ORDER BY order_date) AS running_total,
AVG(revenue) OVER (
ORDER BY order_date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS moving_avg_7day
FROM daily_sales;
```

KEY IDEA: PARTITION BY resets the calculation per group (like a GROUP BY that doesn't collapse rows). ORDER BY inside OVER() controls running/ranking logic, and is completely independent of the query's outer ORDER BY.

Indexes

An index is a separate, sorted data structure (usually a B-tree) that lets the engine find rows without scanning the whole table. Indexes speed up reads but slow down writes (every INSERT/UPDATE must also update the index).

```
CREATE INDEX idx_employees_department ON employees(department);
CREATE UNIQUE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_cust_date ON orders(customer_id, order_date);
```

Good candidates for an index	Poor candidates
Columns in WHERE, JOIN, ORDER BY	Columns rarely filtered/sorted on
High-cardinality columns (many distinct values)	Low-cardinality (e.g., boolean flags)
Foreign key columns	Tables with heavy write volume + few reads

Reading an execution plan

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 12345;
```

Plan term	What it means
Seq Scan / Full Table Scan	Reading every row — red flag on large tables
Index Scan / Index Seek	Using an index to jump straight to matching rows
Nested Loop	Fine for small tables, expensive when both sides are large
Hash Join / Merge Join	Usually efficient for large table × large table joins
Rows estimated vs actual	Big mismatch → stale statistics, run ANALYZE

Optimization checklist

- Filter as early as possible; avoid functions on indexed columns in WHERE (e.g. WHERE YEAR(date)=2024 blocks index use — use a range instead).
- Select only the columns you need — avoid SELECT * in production queries.
- Avoid SELECT DISTINCT / GROUP BY as a substitute for fixing a bad join that duplicates rows.
- Prefer EXISTS over IN for large subqueries; it can short-circuit on first match.
- Batch large UPDATE/DELETE operations instead of one massive transaction.
- Watch for implicit type conversions in WHERE (comparing INT column to a string literal) — they silently disable indexes.

Transactions & ACID

Property	Guarantee
Atomicity	All statements in the transaction succeed, or none do
Consistency	Database moves only between valid states
Isolation	Concurrent transactions don't corrupt each other's view of data
Durability	Once committed, changes survive a crash

```

BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT; -- or ROLLBACK; if anything looks wrong before committing

SAVEPOINT before_risky_step;
-- ... risky statement ...
ROLLBACK TO before_risky_step; -- undo just that part

```

Isolation levels (lowest to highest)

Level	Trade-off
READ UNCOMMITTED	Fastest, allows dirty reads (rare in practice)
READ COMMITTED	Default in Postgres/SQL Server — no dirty reads
REPEATABLE READ	Default in MySQL/InnoDB — no dirty or non-repeatable reads
SERIALIZABLE	Strictest, fully isolated, most locking overhead

Views vs Materialized Views

```

-- Regular view: a saved query, always live, no storage of its own
CREATE VIEW active_customers AS
SELECT * FROM customers WHERE status = 'active';

-- Materialized view: stores results physically, must be refreshed
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS revenue
FROM orders GROUP BY 1;

REFRESH MATERIALIZED VIEW monthly_revenue;

```

Stored procedures & functions (concept)

```

CREATE FUNCTION get_customer_tier(spend NUMERIC)
RETURNS VARCHAR AS $$
BEGIN
IF spend > 10000 THEN RETURN 'Platinum';
ELSIF spend > 5000 THEN RETURN 'Gold';
ELSE RETURN 'Standard';
END IF;
END; $$ LANGUAGE plpgsql;

```

Syntax for procedures/functions varies significantly by engine (PL/pgSQL, T-SQL, PL/SQL) — the logic concepts (parameters, control flow, return values) transfer, the syntax does not.

These recurring problem shapes show up in almost every SQL interview (and in almost every real analytics job). Recognize the pattern, not just the exact wording.

1. Find duplicate rows

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

2. Delete duplicates, keep one copy

```
DELETE FROM users
WHERE user_id NOT IN (
  SELECT MIN(user_id) FROM users GROUP BY email
);
```

3. Nth highest value (per group)

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 2; -- second-highest salary, ties handled correctly
```

4. Running total & cumulative percentage

```
SELECT month, revenue,
SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue,
SUM(revenue) OVER (ORDER BY month) * 100.0 /
SUM(revenue) OVER () AS pct_of_total
FROM monthly_revenue;
```

5. Consecutive days / gaps-and-islands

```
-- Group consecutive login dates into streaks
SELECT user_id, login_date,
login_date - (ROW_NUMBER() OVER (
  PARTITION BY user_id ORDER BY login_date
))::int AS streak_group
FROM logins;
-- Rows with the same streak_group value are one consecutive run
```

6. Pivot rows into columns

```
SELECT
product_id,
SUM(CASE WHEN quarter = 'Q1' THEN revenue ELSE 0 END) AS q1,
SUM(CASE WHEN quarter = 'Q2' THEN revenue ELSE 0 END) AS q2
FROM sales
GROUP BY product_id;
```

7. Year-over-year / month-over-month growth

```
SELECT month, revenue,
LAG(revenue, 12) OVER (ORDER BY month) AS revenue_last_year,
(revenue - LAG(revenue, 12) OVER (ORDER BY month))
/ LAG(revenue, 12) OVER (ORDER BY month) * 100.0 AS yoy_growth_pct
FROM monthly_revenue;
```

String functions

Function	Purpose
CONCAT(a, b) / a b	Join strings
UPPER() / LOWER()	Change case
TRIM() / LTRIM() / RTRIM()	Remove whitespace
SUBSTRING(str, start, len)	Extract part of a string
LENGTH() / CHAR_LENGTH()	String length
REPLACE(str, from, to)	Replace substring
SPLIT_PART(str, delim, n)	Get nth piece after splitting

Date/time functions

Function	Purpose
CURRENT_DATE / NOW()	Today's date / current timestamp
DATE_TRUNC('month', col)	Round date down to start of period
DATEDIFF / col1 - col2	Difference between two dates
EXTRACT(YEAR FROM col)	Pull out a date part
DATE_ADD / col + INTERVAL '1 day'	Add/subtract time

Numeric functions

Function	Purpose
ROUND(n, decimals)	Round a number
CEIL() / FLOOR()	Round up / down to integer
ABS()	Absolute value
MOD(a, b) / a % b	Remainder
POWER(a, b) / SQRT(a)	Exponent / square root

Operator precedence (high to low, simplified)

```

1. Parentheses ()
2. Arithmetic: * / % then + -
3. Comparison: = <> < > <= >=
4. NOT
5. AND
6. OR
-- Always use parentheses in mixed AND/OR conditions – don't rely on memory

```

Query writing checklist before you hit run

- Does every table in a JOIN have an explicit ON condition? (missing one silently creates a cross join)
- Are NULLs handled explicitly where they could appear (COALESCE, IS NULL)?
- For UPDATE/DELETE: have you tested the WHERE clause as a SELECT first?

- Does GROUP BY include every non-aggregated selected column?
- Would EXISTS or a join be faster than IN (subquery) here?
- Is DECIMAL used instead of FLOAT for any money values?

You now have the full arc: query anatomy → joins → aggregation → subqueries/CTEs → window functions → performance → interview patterns. Keep this sheet open while you practice on real datasets — fluency comes from writing queries, not re-reading them.